



Code2Viz - 2D Geometry Visualizer

A WPF application for visualizing 2D geometric shapes through C# or F# code execution with animation support.

Overview

Code2Viz is a visual programming environment that lets you write C# or F# code to create, style, and animate 2D geometric shapes on an interactive canvas. It combines a code editor with syntax highlighting, a real-time rendering canvas with zoom and pan capabilities, and a timeline-based animation system with GIF export.

Features

- **Live Preview:** Canvas updates automatically as you type (debounced auto-run)
- **No Draw() Required:** Shapes appear automatically when created
- **Multi-language Support:** Write code in C# or F# with full syntax highlighting
- **Rich Shape Library:** Points, lines, circles, rectangles, ellipses, arcs, polygons, polylines, Bezier curves, splines, regions (curve-bounded areas), text, arrows, and dimension annotations
- **Shape Editing:** Select shapes and drag shape-specific control points (vertices, radius handles, curve controls) with live code sync
- **Properties Panel:** Floating or dockable panel to edit geometry and style properties (color, fill, weight, opacity, visibility, name) with full code sync — changes persist as code lines
- **Animation System:** Create timeline-based animations with draw, move, rotate, flip, and fade effects
- **Interactive Canvas:** Zoom with mouse wheel, pan with middle-click, toggle grid display
- **Export Options:** Save visualizations as PNG images, animated GIFs, or MP4 videos
- **Project Management:** Organize multiple code files into projects with tabbed editing
- **NuGet Integration:** Add external packages to extend functionality
- **Built-in Help:** Comprehensive API documentation with examples
- **Code Minimap:** VSCode-style minimap with syntax coloring, viewport indicator, and error marker navigation

Quick Start

1. Create a New Project

File > New Project (Ctrl+Shift+N) creates a new project with a starter template.

2. Write Your Code

The entry point is `StartViz.Viz.Main()` in `StartViz.vizcode`:

```
using Code2Viz.Geometry;

namespace StartViz
{
    public class Viz
    {
        public static void Main()
        {
            // Shapes appear automatically when created - no Draw() needed!
            var circle = new VCircle(0, 0, 50);
            circle.Color = "Cyan";
            circle.FillColor = "#4000FFFF";
        }
    }
}
```

3. See Results Instantly

With **Auto-update Canvas** enabled (default), the canvas updates automatically as you type - no need to press Run!

- **Auto-update:** Canvas refreshes 500ms after you stop typing
- **Manual mode:** Disable auto-update in Settings to use F5/Run button instead
- **Draw() is optional:** Shapes appear when created; `Draw()` is kept for backwards compatibility

Supported Shapes

Shape	Description	Constructor Examples
VPoint	A point marker	<code>new VPoint(x, y)</code>
VLine	A line segment	<code>new VLine(p1, p2)</code> or <code>new VLine(x1, y1, x2, y2)</code>
VXLine	An infinite construction line	<code>new VXLine(basePoint, direction)</code> or <code>new VXLine(p1, p2)</code>
VRay	A semi-infinite ray	<code>new VRay(origin, direction)</code> or <code>new VRay(origin, throughPoint)</code>
VCircle	A circle	<code>new VCircle(center, radius)</code> or <code>new VCircle(x, y, radius)</code> or <code>new VCircle(p1, p2, p3)</code> (circumcircle)
VRectangle	A rectangle (inherits from VPolygon)	<code>new VRectangle(corner, width, height)</code> or <code>new VRectangle(bottomLeft, topRight)</code>

Shape	Description	Constructor Examples
VEllipse	An ellipse	<code>new VEllipse(center, radiusX, radiusY)</code>
VArc	A circular arc	<code>new VArc(center, radius, startAngle, endAngle)</code>
VPolygon	A closed polygon	<code>new VPolygon(p1, p2, p3, ...)</code>
VPolyline	Open connected segments	<code>new VPolyline(p1, p2, p3, ...)</code>
VBezier	Cubic Bezier curve	<code>new VBezier(start, ctrl1, ctrl2, end)</code>
VSpline	Smooth spline curve	<code>new VSpline(p1, p2, p3, ...)</code>
VText	Text at a position	<code>new VText(position, "text")</code> or <code>new VText(x, y, "text", height)</code>
VArrow	Arrow with head	<code>new VArrow(start, end)</code>
VDimension	Dimension annotation	<code>new VDimension(p1, p2)</code>
VGroup	Group of shapes	<code>new VGroup(shape1, shape2, ...)</code> or <code>new VGroup(shapeList)</code>
VGrid	Grid of points	<code>new VGrid(location, xcount, ycount, spacing, centered)</code>
Region	Curve-bounded region	<code>new Region(curves)</code> or <code>new Region(outerCurves, holes)</code>

Shape Grouping (VGroup)

VGroup allows you to combine multiple shapes into a single unit that can be transformed and selected together.

Creating Groups

```
// Empty group, add shapes later
var group = new VGroup();
group.Add(new VCircle(0, 0, 20));
group.Add(new VLine(-30, 0, 30, 0));

// From params
var group2 = new VGroup(
    new VCircle(0, 0, 20),
    new VLine(-30, 0, 30, 0),
    new VLine(0, -30, 0, 30)
);

// From collection
```

```
var shapes = new List<Shape> { circle, line1, line2 };  
var group3 = new VGroup(shapes);
```

Group Transformations

All transformations apply to every shape in the group:

```
var group = new VGroup(circle, line1, line2);  
  
// Move entire group  
group.Move(new VXYZ(100, 50, 0));  
  
// Rotate around a pivot point  
group.Rotate(new VPoint(0, 0), 45);  
  
// Scale from center  
group.Scale(group.GetCenter(), 2.0);  
  
// Draw the group (renders as a single selectable entity)  
group.Draw();
```

Group Styling

Apply styles to all shapes at once:

```
var group = new VGroup(shape1, shape2, shape3);  
group.Color = "Cyan";  
group.FillColor = "#4000FFFF";  
group.StrokeThickness = 2;  
  
// Apply group style to all children  
group.ApplyStyle();  
  
// Or apply individual properties  
group.ApplyColor();  
group.ApplyFillColor();  
group.ApplyStrokeThickness();  
  
// Set opacity for all shapes  
group.SetOpacity(0.5);
```

Group Utilities

```
// Access shapes  
int count = group.Count;  
Shape first = group[0];  
bool hasCircle = group.ContainsShape(myCircle);
```

```
// Query shapes by type
var allCircles = group.GetShapesOfType<VCircle>();

// Flatten nested groups
List<Shape> allShapes = group.Flatten();

// Iterate with action
group.ForEach(s => s.Color = "Yellow");

// Filter to new group
var filtered = group.Where(s => s is VCircle);

// Get bounds and center
BoundingBox bounds = group.GetBounds();
VPoint center = group.GetCenter();
// bounds.Min, bounds.Max, bounds.Width, bounds.Height, bounds.Center
```

Point Grids (VGrid)

VGrid creates a rectangular grid of VPoints, useful for creating patterns, matrices, or reference grids.

Creating Grids

```
// Centered grid at origin: 5 columns x 3 rows, spacing 10 units
var grid = new VGrid(new VPoint(0, 0), 5, 3, 10, true);
grid.Draw();

// Grid with bottom-left corner at (-100, -50)
var grid2 = new VGrid(new VPoint(-100, -50), 4, 4, 20, false);
grid2.Draw();

// Different X and Y spacing: 15 horizontal, 10 vertical
var grid3 = new VGrid(new VPoint(0, 0), 6, 4, 15, 10, true);
grid3.Draw();
```

Constructor Options

Constructor	Description
VGrid(location, xcount, ycount, centered)	Default spacing of 1.0
VGrid(location, xcount, ycount, spacing, centered)	Uniform spacing
VGrid(location, xcount, ycount, xSpacing, ySpacing, centered)	Different X/Y spacing

Grid Properties

--

```

var grid = new VGrid(new VPoint(0, 0), 5, 3, 10, true);

// Access points
List<VPoint> allPoints = grid.Points;
int totalCount = grid.Count;           // 15 (5 x 3)
VPoint point = grid[0];                 // First point (by index)
VPoint cell = grid[2, 1];               // Column 2, Row 1

// Grid info
int cols = grid.XCount;                 // 5
int rows = grid.YCount;                 // 3
double xSpace = grid.XSpacing;          // 10
double ySpace = grid.YSpacing;          // 10
bool centered = grid.Centered;          // true

```

Grid Operations

```

var grid = new VGrid(new VPoint(0, 0), 5, 3, 10, true);

// Style all points
grid.Color = "White";
grid.FillColor = "Cyan";
grid.ApplyStyle(); // Apply to all points

// Get rows and columns
List<VPoint> row0 = grid.GetRow(0);      // Bottom row
List<VPoint> col2 = grid.GetColumn(2);   // Third column

// Geometry
VPoint center = grid.GetCenter();
BoundingBox bounds = grid.GetBounds();
// bounds.Min, bounds.Max, bounds.Width, bounds.Height

// Transform entire grid
grid.Move(new VXYZ(50, 25, 0));
grid.Rotate(new VPoint(0, 0), 45);
grid.Scale(grid.GetCenter(), 2.0);

grid.Draw();

```

Regions (Curve-Bounded Areas)

Region represents an enclosed 2D area bounded by curves (lines, arcs, splines, beziers). Unlike VPolygon which only supports straight edges, Region preserves the original curve geometry in its boundary loops.

Creating Regions

```
// Region from lines (rectangle)
var p0 = VPoint.Internal(0, 0);
var p1 = VPoint.Internal(100, 0);
var p2 = VPoint.Internal(100, 80);
var p3 = VPoint.Internal(0, 80);

var region = new Region(new List<ICurve> {
    new VLine(p0, p1),
    new VLine(p1, p2),
    new VLine(p2, p3),
    new VLine(p3, p0)
});
region.Color = "Cyan";
region.FillColor = "#4000FFFF";

// Region with mixed curves (D-shape: line + arc)
var bottom = VPoint.Internal(0, 0);
var top = VPoint.Internal(0, 60);
var arc = VArc.FromStartEndRadius(top, bottom, 40, false);
var dShape = new Region(new List<ICurve> { new VLine(bottom, top), arc });

// Curves can be provided in any order - they are auto-ordered into a loop
```

Regions with Holes

```
// Create outer boundary
var outer = new Region(new List<ICurve> {
    new VLine(VPoint.Internal(0,0), VPoint.Internal(100,0)),
    new VLine(VPoint.Internal(100,0), VPoint.Internal(100,100)),
    new VLine(VPoint.Internal(100,100), VPoint.Internal(0,100)),
    new VLine(VPoint.Internal(0,100), VPoint.Internal(0,0))
});

// Add a hole
outer.AddHole(new List<ICurve> {
    new VLine(VPoint.Internal(30,30), VPoint.Internal(70,30)),
    new VLine(VPoint.Internal(70,30), VPoint.Internal(70,70)),
    new VLine(VPoint.Internal(70,70), VPoint.Internal(30,70)),
    new VLine(VPoint.Internal(30,70), VPoint.Internal(30,30))
});

// Or provide holes in constructor
var regionWithHoles = new Region(outerCurves, new List<List<ICurve>> { holeCurves
});
```

Region Properties

```

var region = new Region(curves);

double area = region.Area;           // Outer area minus hole areas
double signed = region.SignedArea;   // Positive for CCW, negative for CW
double perimeter = region.Perimeter; // Total length (outer + holes)

List<ICurve> outer = region.OuterLoop; // Outer boundary curves
List<List<ICurve>> holes = region.Holes; // Inner hole loops

bool inside = region.Contains(VPoint.Internal(50, 40)); // Point containment
BoundingBox bounds = region.GetBounds();

```

Converting Between Region and Polygon

```

// Region to Polygon
var poly = region.ToPolygon();           // Low-fidelity (curve endpoints only)
var hires = region.ToPolygonHighRes(32); // High-fidelity (32 segments per curve)
var pwh = region.ToPolygonWithHoles(32); // With holes, high-fidelity

// Polygon to Region
var fromPoly = Region.FromPolygon(polygon);
var fromPwh = Region.FromPolygonWithHoles(polygonWithHoles);

```

Region Boolean Operations

Region supports boolean operations via `RegionBooleanOps` or extension methods:

```

// Static methods
var union = RegionBooleanOps.Union(regionA, regionB);           // Region?
var intersection = RegionBooleanOps.Intersect(regionA, regionB); // List<Region>
var difference = RegionBooleanOps.Difference(regionA, regionB); // List<Region>
var xor = RegionBooleanOps.Xor(regionA, regionB);                // List<Region>

// Multi-region union
var combined = RegionBooleanOps.Union(region1, region2, region3);

// Extension method syntax
var union = regionA.Union(regionB);
var diff = regionA.Difference(regionB);

// With holes support
var results = RegionBooleanOps.DifferenceWithHoles(regionA, regionB);

```

Note: Use `RegionBooleanOps.Intersect(a, b)` instead of `a.Intersect(b)` to avoid collision with `Shape.Intersect`.

Shape Styling

All shapes support customizable styling through these properties:

```
var circle = new VCircle(0, 0, 50);
circle.Color = "Cyan";           // Outline color
circle.FillColor = "#4000FFFF";  // Fill color (with transparency)
circle.StrokeThickness = 2.5;    // Border thickness
circle.StrokeStyle = StrokeStyle.Dashed; // Line pattern
circle.Draw();
```

Stroke Styles

The `StrokeStyle` property controls the line pattern for shape outlines:

Style	Description	Pattern
Continuous	Solid line (default)	_____
Dashed	Long dashes	— — —
Dotted	Short dots	
DashDot	Dash-dot alternating	— . — .
DashDotDot	Dash-dot-dot pattern	— . . —
Center	Center line (long-short)	— — — —
Phantom	Phantom line	— — — —
Hidden	Hidden line (short dashes)	- - - - -

```
var line1 = new VLine(0, 0, 100, 0);
line1.StrokeStyle = StrokeStyle.Dashed;
line1.Draw();

var line2 = new VLine(0, 20, 100, 20);
line2.StrokeStyle = StrokeStyle.DashDot;
line2.Draw();
```

Color Formats

- **Named colors:** "Red", "Blue", "Cyan", "LimeGreen", etc.
- **Hex RGB:** "#FF0000" (red)
- **Hex ARGB:** "#80FF0000" (semi-transparent red, where 80 is alpha)

VColor Utility Class

Use `VColor` for easy color access and random color generation:

```
// Static color properties
circle.Color = VColor.Red;
circle.FillColor = VColor.LimeGreen;

// Random colors
shape.Color = VColor.GetRandomColor();           // pastel (default)
shape.Color = VColor.GetRandomColor(false);      // vibrant
shape.FillColor = VColor.GetRandomPastelColor();  // shorthand for pastel
shape.Color = VColor.GetRandomVibrantColor();    // shorthand for vibrant

// Custom RGB colors
shape.FillColor = VColor.FromRgb(255, 128, 0);    // orange
shape.FillColor = VColor.FromArgb(128, 255, 0, 0); // semi-transparent red

// From enum
shape.Color = VColor.FromEnum(ColorName.Coral);
```

Color categories:

- **Vibrant colors** (25): Bright colors good for strokes - Red, Lime, Cyan, HotPink, Gold, etc.
- **Pastel colors** (25): Soft colors good for fills - LightBlue, Lavender, PaleGreen, etc.

Global Defaults

Set default styling for all new shapes:

```
ShapeDefaults.GlobalColor = "Cyan";
ShapeDefaults.GlobalFillColor = "Transparent";
ShapeDefaults.GlobalStrokeThickness = 2.0;
ShapeDefaults.GlobalStrokeStyle = StrokeStyle.Continuous;

// All shapes created after this use these defaults
var circle = new VCircle(0, 0, 50);
circle.Draw(); // Uses Cyan stroke

// Reset to original defaults
ShapeDefaults.Reset();
```

Geometric Properties

Circles and ellipses provide computed geometric properties:

```
// Circle properties
var circle = new VCircle(0, 0, 50);
double area = circle.Area;           //  $\pi \times r^2 = \sim 7853.98$ 
double circumference = circle.Circumference; //  $2\pi \times r = \sim 314.16$ 

// Ellipse properties
```

```
var ellipse = new VEllipse(0, 0, 60, 40);
double ellipseArea = ellipse.Area;           //  $\pi \times rx \times ry = \sim 7539.82$ 
double ellipseCircum = ellipse.Circumference; // Ramanujan approximation =
~318.49
```

Animation System

Code2Viz includes an animation system using the **Animator** class for creating animated visualizations with automatic sequencing.

Note: The animation timeline panel is automatically hidden when your code has no animations. It appears automatically when you run code that creates animations.

Basic Animation Example

```
using Code2Viz.Geometry;
using Code2Viz.Animation;

namespace StartViz
{
    public class Viz
    {
        public static void Main()
        {
            // Create shapes
            var line = new VLine(0, 0, 100, 50);
            var circle = new VCircle(50, 50, 30);
            circle.Color = "Yellow";

            // Create animator
            var anim = new Animator();
            anim.Repeat = true; // Loop animation
            anim.Fps = 30;     // Limit to 30 frames per second (1-120, default
60)

            // Add animations - they play sequentially
            anim.AddToAnimations(new DrawAnimation(line, 2.0)); // 0-2s
            anim.AddToAnimations(new DrawAnimation(circle, 2.0)); // 2-4s
            anim.AddToAnimations(new MoveAnimation(circle, new VXYZ(50, 0, 0),
2.0)); // 4-6s

            // Start playback
            anim.Animate();
        }
    }
}
```

Parallel Animations

Add multiple animations as a list to play them simultaneously:

```
var anim = new Animator();

// These play in parallel (both start at 0s, both last 2s)
anim.AddToAnimations(new List<Animation> {
    new FadeInAnimation(shape1, 2.0),
    new FadeInAnimation(shape2, 2.0)
});

// This plays after the parallel group finishes (starts at 2s)
anim.AddToAnimations(new DrawAnimation(line, 1.0)); // 2-3s

anim.Animate();
```

Animation Types

Animation	Description	Constructor
DrawAnimation	Progressive drawing (0% to 100%)	new DrawAnimation(shape, duration)
MoveAnimation	Move by displacement vector	new MoveAnimation(shape, displacement, duration)
RotateAnimation	Rotate around a pivot point	new RotateAnimation(shape, pivot, angleDegrees, duration)
FlipAnimation	Mirror across an axis line	new FlipAnimation(shape, mirrorAxis, duration)
FadeInAnimation	Fade from transparent to opaque	new FadeInAnimation(shape, duration)
FadeOutAnimation	Fade from opaque to transparent	new FadeOutAnimation(shape, duration, targetOpacity)
ValueAnimation<T>	Animate any numeric property on a shape	new ValueAnimation<VCircle>(circle, c => c.Radius, 0, 50, 3.0)
ObjectPropertyAnimation<T>	Animate any numeric property on any object	new ObjectPropertyAnimation<Wheel>(wheel, w => w.Rotation, 0, 360, 1.0)

ObjectPropertyAnimation Example

ObjectPropertyAnimation works like ValueAnimation but targets any object, not just shapes. This is useful for animating properties on user-defined classes:

```

public class Wheel
{
    VCircle c = new VCircle(0, 0, 100);
    VCircle hub = new VCircle(new VPoint(40, 40), 10);

    private double rotation = 0.0;
    public double Rotation
    {
        get { return rotation; }
        set { set_rotation(value); rotation = value; }
    }

    private void set_rotation(double value)
    {
        hub.Rotate(new VPoint(0, 0), value - rotation);
    }
}

// In Main():
var wheel = new Wheel();
var anim = new Animator();
anim.AddToAnimations(new ObjectPropertyAnimation<Wheel>(wheel, w => w.Rotation,
0.0, 359.0, 1));
anim.Repeat = true;
anim.Animate();

```

Pausing Between Animations

Insert a time gap between sequential animations:

```

var anim = new Animator();

anim.AddToAnimations(new DrawAnimation(line, 2.0)); // 0-2s
anim.Pause(5); // 2-7s: nothing happens
anim.AddToAnimations(new DrawAnimation(circle, 2.0)); // 7-9s

anim.Animate();

```

Easing Functions

Smooth animations with built-in easing:

```

var anim = new MoveAnimation(shape, displacement, start, duration);
anim.EasingFunction = EasingFunctions.EaseInOutQuad; // Smooth start and end

```

Available Easing Functions

Function	Formula	Effect
Linear	t	Constant speed
EaseInQuad	t ²	Slow start, accelerates
EaseOutQuad	t(2-t)	Fast start, decelerates
EaseInOutQuad	Piecewise	Slow start & end
EaseInCubic	t ³	Slower start
EaseOutCubic	(t-1) ³ +1	Slower end
EaseInOutCubic	Piecewise	Smooth start & end

Console Output

Use **VizConsole** to output debug messages:

```
VizConsole.Log("Starting visualization...");
VizConsole.Log($"Circle radius: {circle.Radius}");

// Collections are itemized by default
var nums = new List<int> { 1, 2, 3 };
VizConsole.Log(nums);           // Prints each item on its own line
VizConsole.Log(nums, false);    // Prints
"System.Collections.Generic.List`1[System.Int32]"

// Empty collections show "(empty)" instead of no output
var empty = new List<int>();
VizConsole.Log(empty);          // Prints "(empty)"
```

Output appears in the console panel below the canvas with file and line number tracking:

```
[StartViz:15] Starting visualization...
[StartViz:16] Circle radius: 50
[StartViz:19] 1
[StartViz:19] 2
[StartViz:19] 3
```

Canvas Features

Interactive Controls

- **Mouse Wheel:** Zoom in/out centered on cursor position
- **Middle-Click Drag:** Pan the canvas view
- **Grid Toggle:** Show/hide reference grid lines (View menu)

- **Auto Zoom Extents:** Automatically fits all shapes after execution

Coordinate System

Code2Viz uses a **mathematical coordinate system**:

- Origin (0, 0) is at the center of the canvas
- X-axis increases to the right (+X = right)
- Y-axis increases upward (+Y = up, not down like screen coordinates)
- Angles are measured in degrees, counter-clockwise from the positive X-axis

Shape Editing

Selecting Shapes

- **Click** a shape on the canvas to select it
- **Shift+Click** to add to selection
- **Ctrl+Click** to toggle selection
- **Drag right** on empty area for **Window Selection** (blue solid box, selects shapes fully inside)
- **Drag left** on empty area for **Crossing Selection** (green dashed box, selects shapes that intersect)
- **Ctrl+A** to select all shapes
- **Escape** to deselect

Control Points

When a shape is selected, control point handles appear for interactive editing. Each shape type has specific control points:

Shape	Control Points
VPoint	Move handle at position
VLine	Move at midpoint, vertices at start/end
VCircle	Move at center, radius handle
VArc	Move at center, radius handle, vertices at start/end angles
VRectangle	Move at center, vertices at corners
VEllipse	Move at center, RadiusX and RadiusY handles
VPolygon	Move at centroid, vertex at each point
VPolyline	Move at centroid, vertex at each point
VBezier	Move at midpoint, vertices at P0/P3, curve controls at P1/P2
VSpline	Move at centroid, curve control at each point
VArrow	Move at midpoint, vertices at start/end
VText	Move at location

Shape	Control Points
VDimension	Move at midpoint, vertices at Point1/Point2

Drag any control point to edit the shape geometry. The source code updates automatically when you release.

Properties Panel

Open via **Windows > Properties** menu. The panel shows:

- **Shape info:** Type, ID, and editable Name
- **Geometry:** Shape-specific numeric properties (coordinates, radii, dimensions)
- **Style:** Color and fill color (with color picker), line weight slider, opacity slider, visibility toggle

The panel can be **floated** as a separate window or **docked** to the right side of the main window using the Dock/Float button in the panel header. Multi-selection shows common style properties.

Drawing Tools

Code2Viz includes an interactive drawing toolbar that lets you create shapes directly on the canvas with automatic C#/F# code generation.

Toolbar Location

The drawing toolbar appears below the menu bar with buttons for all shape types.

Drawing Methods

Shape	Method	Clicks
Point	Single click	1
Line	Click start, click end	2
Circle	Click center, click radius point	2
Rectangle	Click corner, click opposite corner	2
Ellipse	Click center, drag for radii	2
Arc	Click center, click start, click end	3
Polygon	Click vertices, double-click to close	N + double-click
Polyline	Click points, double-click to finish	N + double-click
Bezier	Click start, ctrl1, ctrl2, end	4
Spline	Click control points, double-click	N + double-click
Arrow	Click start, click end	2
Text	Click position	1

Snap Support

While drawing, the tool automatically snaps to various geometric features. Visual indicators show snap points as you move the cursor.

Basic Snap Types

Snap Type	Marker	Description
Endpoint	Yellow square	Start/end points of lines, arcs, polylines
Midpoint	Cyan triangle	Middle point of lines and curves
Center	Magenta circle	Center of circles, arcs, ellipses
Intersection	Red X	Where two shapes cross
Nearest	Green diamond	Closest point on any curve

Advanced Snap Types

Extension Snap

When placing the second point (or subsequent points), the **Extension** snap shows a dotted line extending from endpoints of existing lines, polylines, polygons, and rectangles.

- **Visual:** A dotted cyan line extends along the direction of the edge
- **Label:** Shows "Extension: [distance] < [angle]°" with the distance from the endpoint and the angle
- **Magnetic Effect:** The cursor stays snapped to the extension line within a tolerance, allowing you to draw precise aligned lines
- **Reach:** Extension lines are detected up to 300 pixels from the source endpoint

Perpendicular Snap

When picking the second point, the **Perpendicular** snap shows the point that creates a perpendicular relationship from your first click to an existing line or curve.

- **Visual:** An orange dotted line from your first point to the perpendicular point on the target shape
- **Use Case:** Perfect for drawing lines at 90° to existing geometry

Tangent Snap

When picking the second point near a circle or arc, the **Tangent** snap shows the tangent point where a line from your first click would touch the circle.

- **Visual:** A violet dotted line from your first point to the tangent point on the circle/arc
- **Use Case:** Drawing lines that touch circles at exactly one point

Precise Distance and Angle Input

While drawing (after placing the first point), you can type precise values for distance and angle instead of clicking.

How to Use

1. **Start drawing** (e.g., Line tool) and click to place the first point
2. **Move cursor** over the canvas - you'll see the preview line
3. **Type a number** (e.g., "100") - Distance input mode activates automatically
 - The current distance is shown pre-selected; typing replaces it
4. **Press Tab** to switch to Angle input mode
5. **Type the angle** in degrees (e.g., "45")
6. **Press Enter** to place the point at the specified distance and angle
7. **Press Escape** to cancel input mode

Input Mode Indicators

- When typing distance: `Extension: [100_] < 45°` (brackets show active field)
- When typing angle: `Extension: 100.00 < [45_]°`

Keys in Input Mode

Key	Action
0-9, ., -	Type value (first keystroke replaces pre-selected value)
Tab	Cycle through modes: Distance → Angle → None
Backspace	Delete last character (or clear all if value is selected)
Enter	Confirm and place point at specified distance/angle
Escape	Cancel input mode

This feature works for all multi-point drawing tools (Line, Polyline, Polygon, etc.) and enables CAD-style precise drawing without needing to calculate coordinates manually.

Orthogonal Constraint (Shift Key)

When drawing lines, polylines, polygons, splines, arrows, or bezier curves:

- Hold **Shift** after placing the first point to constrain the line to horizontal or vertical
- The constraint automatically chooses the axis with the larger movement
- Status bar shows "(Shift: ortho)" hint when the feature is available
- Works with snap points - the constraint is applied before snapping

Automatic Code Generation

When you complete drawing a shape, the corresponding code is automatically inserted into the `Main()` method of your entry point file:

```
// Generated when you draw a line from (100, 50) to (200, 150)
new VLine(100.00, 50.00, 200.00, 150.00).Draw();

// Generated when you draw a circle at (150, 100) with radius 75.5
new VCircle(150.00, 100.00, 75.50).Draw();
```

Drawing Tool Shortcuts

When the editor is not focused:

Shortcut	Action
P	Point tool
L	Line tool
C	Circle tool
R	Rectangle tool
Shift (hold)	Orthogonal constraint (H/V lock)
Esc	Cancel drawing / Return to select mode

Shape IDs and Outliner

Unique Shape IDs

Every shape has a unique `Id` property (long integer) automatically assigned when created. The ID counter resets on each code execution, so IDs always start from 1:

```
var circle = new VCircle(0, 0, 50);
var line = new VLine(0, 0, 100, 100);
VizConsole.Log($"Circle ID: {circle.Id}"); // 1
VizConsole.Log($"Line ID: {line.Id}");    // 2
```

Outliner Panel

The Outliner panel (below the Explorer) displays all shapes grouped by type:

- Shows shape count per type: "VCircle (3)"
- Each shape displays its name and clickable ID
- Click an ID to zoom the canvas to that shape
- **Hover over any shape** to highlight it on the canvas with a colored overlay
- Right-click for **Expand All** / **Collapse All** options

Highlight Settings

The Outliner hover highlight can be customized in the Settings tab (Application Settings):

- **Highlight Color:** Choose any color for the highlight overlay (default: Yellow)
- **Highlight Opacity:** Adjust transparency from 10% to 100% (default: 40%)

Zoom To Shape

Use **View > Zoom To Shape** (or **Ctrl+G**) to zoom to a specific shape by entering its ID.

Measuring Tape Tool

Code2Viz includes a precision measuring tool with AutoCAD-style snap features.

Activating the Tool

Press **Ctrl+M** to toggle the Measuring Tape tool. Press **Esc** to cancel.

How to Measure

1. Press **Ctrl+M** to activate the tool
2. Move the mouse - snap indicators appear near snap points
3. **Click first point** (snaps if within tolerance)
4. Move mouse - a dashed measuring line shows with live distance
5. **Click second point** - measurement displayed in status bar
6. Tool stays active for additional measurements; press **Esc** to exit

Snap Types

The measuring and drawing tools support 8 snap types (configurable in Settings):

Snap Type	Marker	Description
Endpoint	Yellow square	Start/end points of lines, arcs, polylines
Midpoint	Cyan triangle	Middle point of lines and curves
Center	Magenta circle	Center of circles, arcs, ellipses
Intersection	Red X	Where two shapes cross
Nearest	Green diamond	Closest point on any curve
Perpendicular	Orange right-angle	Perpendicular from first click point to existing geometry
Extension	Cyan dotted line	Extended line along existing edges
Tangent	Violet line	Tangent point from first click to circles/arcs

Snap Settings

Configure snap behavior in the Settings tab (Application Settings > Snap Settings):

- Toggle each snap type on/off individually
- All 8 snap types can be independently enabled/disabled
- Settings are saved globally and persist across sessions

Export Options

PNG Export

File > Export > PNG (or Ctrl+E) exports the current canvas view as a PNG image.

GIF Export

File > Export GIF Animation exports animations as animated GIF files with options:

- **Duration:** Animation length in seconds (1-30s)
- **Frame Rate:** 5-30 FPS
- **Background:** Current canvas, white, or black
- **Include Grid:** Optionally include grid and axes

Video Export (MP4)

File > Export Video (MP4) exports animations as H.264 MP4 video files. The export renders vector graphics at the target resolution for crisp, sharp output.

Animation Settings

Setting	Range	Description
Duration	1-60 seconds	Length of the exported video
Frame Rate	15, 30, 45, 60 FPS	Higher = smoother motion, larger file
Bitrate	1-20 Mbps	Higher = better quality, larger file

Resolution Presets

Preset	Dimensions	Use Case
Canvas Size	Current window size	Quick export at screen resolution
720p	1280×720	Web/social media, smaller files
1080p	1920×1080	Full HD, good balance of quality/size
4K	3840×2160	Maximum quality, large files
Custom	User-defined	Any resolution from 64 to 4096 pixels

Background Options

- **Current Canvas Background:** Uses your canvas background color
- **White:** Clean white background
- **Black:** Dark background for contrast

Additional Options

- **Include Grid & Axes:** Toggle grid lines in the export

Technical Notes

- Uses Windows Media Foundation H.264 encoder (no external dependencies)
- Renders vectors at target resolution using high DPI for sharp lines and text
- Aspect ratio is preserved; letterbox/pillarbox filled with background color
- Dimensions automatically adjusted to even numbers (H.264 requirement)

DXF Export

File > Export > DXF exports shapes to AutoCAD DXF format (R12 ASCII):

- Compatible with AutoCAD, LibreCAD, and other CAD software
- Supports all shape types (lines, circles, arcs, polygons, text, etc.)
- Preserves geometry with high precision

PDF Export

File > Export > PDF exports shapes to vector PDF format:

- High-quality vector graphics output
- Preserves colors and stroke styles
- Suitable for printing and documentation

SVG Export

File > Export > SVG exports shapes to SVG (Scalable Vector Graphics) format:

- Web-compatible vector format
- Opens in any browser or vector editor (Inkscape, Illustrator)
- XML-based, can be edited as text
- Supports all shape types with full styling

Boolean Operations

Code2Viz provides polygon boolean operations using the Clipper2 library. For curve-bounded regions, see also [Region Boolean Operations](#).

Available Operations

```
var poly1 = new VPolygon(new VPoint(0,0), new VPoint(100,0), new VPoint(100,100),
new VPoint(0,100));
var poly2 = new VPolygon(new VPoint(50,50), new VPoint(150,50), new
VPoint(150,150), new VPoint(50,150));

// Union - combine two polygons (returns single VPolygon or null if disjoint)
var union = poly1.Union(poly2);
union?.Draw();
```

```
// Intersection - get overlapping area
var intersection = poly1.Intersect(poly2);

// Difference - subtract poly2 from poly1
var difference = poly1.Difference(poly2);

// XOR - symmetric difference
var xor = poly1.Xor(poly2);
```

Utility Methods

```
// Check if point is inside polygon
bool inside = polygon.Contains(new VPoint(50, 50));

// Calculate polygon area
double area = polygon.GetArea();

// Offset polygon (positive = outward, negative = inward)
var offsetPolygons = BooleanOps.OffsetPolygon(polygon, 10);

// Simplify polygon (remove redundant points)
var simplified = BooleanOps.Simplify(polygon, tolerance: 0.1);
```

Array/Pattern Operations

Create arrays and patterns of shapes with built-in array operations.

Linear Array

```
var circle = new VCircle(0, 0, 20);

// Array along X axis: 5 copies, 50 units apart
circle.LinearArrayX(5, 50).DrawAll();

// Array along Y axis: 4 copies, 40 units apart
circle.LinearArrayY(4, 40).DrawAll();

// Array along custom direction
circle.LinearArray(new VXYZ(1, 1, 0), 6, 30).DrawAll();
```

Rectangular Array

```
var rect = new VRectangle(0, 0, 30, 20);
```

```
// 3 rows, 4 columns with spacing
rect.RectangularArray(rows: 3, cols: 4, rowSpacing: 40, colSpacing: 50).DrawAll();
```

Circular Array

```
var shape = new VCircle(50, 0, 10);
var center = new VPoint(0, 0);

// 8 copies in full circle
shape.CircularArray(center, count: 8).DrawAll();

// 6 copies spanning 180 degrees
shape.CircularArray(center, count: 6, totalAngleDegrees: 180).DrawAll();
```

Path Array

```
var marker = new VCircle(0, 0, 5);
var path = new VSpline(new VPoint(0,0), new VPoint(50,100), new VPoint(100,0));

// 10 markers along the path, aligned to path direction
marker.PathArray(path, count: 10, alignToPath: true).DrawAll();
```

Spiral Array

```
var dot = new VCircle(0, 0, 3);
var center = new VPoint(0, 0);

// 30 dots in a spiral from radius 20 to 100, 2 revolutions
dot.SpiralArray(center, count: 30, startRadius: 20, endRadius: 100,
totalRevolutions: 2).DrawAll();
```

Mirror

```
var triangle = new VPolygon(new VPoint(0,0), new VPoint(50,0), new VPoint(25,40));
var mirrorAxis = new VLine(0, -50, 0, 50); // Y-axis

// Creates original + mirrored copy
triangle.Mirror(mirrorAxis).DrawAll();
```

Keyboard Shortcuts

Running Code

Shortcut	Action
F5	Run code
Ctrl+Enter	Run code

File Operations

Shortcut	Action
Ctrl+Shift+N	New project
Ctrl+N	New file
Ctrl+O	Open project
Ctrl+S	Save all files

Editor Operations

Shortcut	Action
Ctrl+Shift+F	Format code
Ctrl+/ Tab / Shift+Tab	Toggle comment Indent / Unindent

Find and Replace

Shortcut	Action
Ctrl+F	Open Find dialog
Ctrl+H	Open Find and Replace dialog
F3	Find Next (in dialog)
Shift+F3	Find Previous (in dialog)

Line Operations

Shortcut	Action
Alt+Up	Move line up
Alt+Down	Move line down
Shift+Alt+Up	Copy line up
Shift+Alt+Down	Copy line down
Ctrl+Shift+D	Delete line

Selection Operations

Shortcut	Action
Shift+Alt+Right	Expand selection (word → brackets → line → block)
Shift+Alt+Left	Shrink selection (undo expand)
Ctrl+D	Add next occurrence with multi-cursor support
Ctrl+Shift+L	Select all occurrences
Ctrl+Alt+Up	Add cursor above
Ctrl+Alt+Down	Add cursor below
Esc	Exit multi-cursor mode

Multi-Cursor Editing

Code2Viz supports VS Code-style multi-cursor editing:

1. **Ctrl+D**: Selects word at cursor, then adds next occurrences
2. **Ctrl+Alt+Up/Down**: Adds cursors vertically above/below
3. **Type**: Text is inserted at ALL cursor positions simultaneously
4. **Backspace/Delete**: Works at all cursor positions
5. **Arrow Keys**: Move all cursors (Left/Right/Up/Down)
6. **Home/End**: Move all cursors to line start/end
7. **Shift+Arrow/Home/End**: Extend selections at all cursors
8. **Escape**: Exits multi-cursor mode
9. **Click elsewhere**: Clears all multi-cursors

All cursors are visually indicated with white caret lines, and selections are highlighted.

Canvas & Tools

Shortcut	Action
Mouse Wheel	Zoom
Middle Click	Pan
Double-click (empty space)	Zoom to fit all shapes
Ctrl+G	Zoom to shape by ID
Ctrl+M	Toggle Measuring Tape tool
F4	Toggle Properties panel
F9	Toggle Snap to Grid
Ctrl+Shift+M	Toggle Minimap
Esc	Cancel current tool/operation

Code Navigation & Intellisense

Shortcut	Action
F12	Go to Definition
Shift+F12	Find All References
Alt+F12	Peek Definition
Ctrl+.	Quick Fix (add missing using)
Ctrl+Shift+O	Document Symbols (outline)
Ctrl+T	Workspace Symbols (search all files)
Ctrl+Shift+H	Call Hierarchy
Ctrl+Shift+T	Type Hierarchy
F2	Rename Symbol

Intellisense & Code Editor Features

Code2Viz includes a full-featured code editor with VSCode-like intellisense powered by Roslyn.

Autocomplete

- **Automatic:** Triggered on typing `.` or `Ctrl+Space`
- **Smart suggestions:** Context-aware completions for types, members, keywords
- **Signature Help:** Parameter info displayed when typing method calls
- **Snippets:** Code snippets for common patterns (if, for, foreach, etc.)

Code Navigation

Feature	Shortcut	Description
Go to Definition	F12	Jump to the definition of a symbol
Peek Definition	Alt+F12	View definition in an inline popup without leaving current location
Find All References	Shift+F12	Find all usages of a symbol (results in console)
Document Symbols	Ctrl+Shift+O	Quick outline of current file (classes, methods, properties)
Workspace Symbols	Ctrl+T	Search symbols across all project files

Code Analysis

Feature	Description
Error Squiggles	Real-time error highlighting with red underlines
Hover Tooltips	Documentation and type info on mouse hover

Feature	Description
Quick Fixes	<code>Ctrl+.</code> suggests fixes like adding missing <code>using</code> statements

Refactoring

Feature	Shortcut	Description
Rename	<code>F2</code>	Rename symbol across all usages
Format Document	<code>Ctrl+Shift+F</code>	Auto-format entire document
Format on Type	Automatic	Formats line when typing <code>;</code> or <code>}</code>

Find and Replace

Feature	Shortcut	Description
Find	<code>Ctrl+F</code>	Search in current file
Find and Replace	<code>Ctrl+H</code>	Search and replace in current file
Find in Files	Edit menu	Search across all project files
Find Next	<code>F3</code>	Jump to next match
Find Previous	<code>Shift+F3</code>	Jump to previous match

Search Options:

- **Case sensitive:** Match exact case
- **Whole word:** Match complete words only
- **Regular expressions:** Use regex patterns for advanced searches

Find Results Panel:

- Results displayed in a tabbed panel below the canvas
- Click any result to navigate to that location
- Results show file name, line number, and matching text

Advanced Features

Feature	Toggle	Description
Semantic Highlighting	View menu	Colors identifiers by semantic meaning (parameters, fields, methods, types)
Inlay Hints	View menu	Shows parameter names and inferred types inline
Code Lens	View menu	Shows reference counts above methods and types
Breadcrumb Navigation	Always on	Shows current location (namespace > class > method) at top of editor

Feature	Toggle	Description
Call Hierarchy	Ctrl+Shift+H	Shows callers and callees of a method
Type Hierarchy	Ctrl+Shift+T	Shows base types and derived types

Semantic Highlighting Colors

When enabled, identifiers are colored based on their meaning:

- **Light Blue**: Local variables, parameters, fields, properties
- **Light Yellow**: Methods
- **Teal**: Classes, structs
- **Light Green**: Interfaces, enums, type parameters
- **Cyan**: Constants, enum members, static fields

Project Structure

File Format

Code2Viz projects use `.vizcode` files. All files in the same directory (and subdirectories) are compiled together.

Entry Point

The entry point must be `StartViz.Viz.Main()` in `StartViz.vizcode`:

```
namespace StartViz
{
    public class Viz
    {
        public static void Main()
        {
            // Your code here
        }
    }
}
```

Available Namespaces

```
using System;
using System.Collections.Generic;
using System.Linq;
using Code2Viz.Geometry;    // Shapes: VPoint, VLine, VCircle, etc.
using Code2Viz.Animation;  // Timeline, DrawAnimation, MoveAnimation, etc.
using Code2Viz.Console;    // VizConsole.Log()
```

NuGet Package Manager

Code2Viz includes a built-in NuGet Package Manager to add external libraries to your projects.

Opening the Package Manager

Tools > NuGet Package Manager (or use the toolbar button)

Features

- **Search:** Search the NuGet repository for packages by name
- **Install:** Select a package and version, then click Install to add it to your project
- **Update:** If a newer version is available, you can update existing packages
- **Remove:** Remove packages you no longer need

Using Installed Packages

After installing a package, add its namespace to your code:

```
using Newtonsoft.Json; // Example: after installing Newtonsoft.Json

namespace StartViz
{
    public class Viz
    {
        public static void Main()
        {
            var obj = new { Name = "Test", Value = 42 };
            var json = JsonConvert.SerializeObject(obj);
            VizConsole.Log(json);
        }
    }
}
```

Package Storage

Packages are stored in a **.packages** folder within your project directory. This folder is created automatically when you install your first package.

Geometry Utilities

VXYZ - 3D Vector

```
var v = new VXYZ(10, 20, 0);
double length = v.GetLength();
var normalized = v.Normalize();
var cross = v1.CrossProduct(v2);
var dot = v1.DotProduct(v2);
```

```
// Rotate a vector around the Z-axis
var rotated = v.Rotate(90); // Returns new VXYZ rotated 90 degrees

// Static basis vectors
var x = VXYZ.BasisX; // (1, 0, 0)
var y = VXYZ.BasisY; // (0, 1, 0)
var z = VXYZ.BasisZ; // (0, 0, 1)
```

Common Shape Methods

All shapes inherit from **Shape** and support these methods:

```
// Shapes appear automatically - Draw() is optional (kept for backwards
compatibility)
var copy = shape.Clone(); // Create a copy
shape.Move(new VXYZ(10, 20, 0)); // Translate
shape.Rotate(pivot, 45); // Rotate 45 degrees around pivot
shape.Scale(center, 2.0); // Scale by factor
BoundingBox bounds = shape.GetBounds(); // Get bounding box
// bounds.Min, bounds.Max, bounds.Width, bounds.Height, bounds.Center, bounds.Area
bool hit = shape.Contains(point); // Point containment test
double d = shape.DistanceTo(pt); // Distance to point
shape.Hide(); // Hide shape from canvas
shape.Show(); // Show hidden shape
```

BoundingBox

The **GetBounds()** method returns a **BoundingBox** object with Min and Max corner points:

```
BoundingBox bounds = shape.GetBounds();
VPoint min = bounds.Min; // Lower-left corner
VPoint max = bounds.Max; // Upper-right corner
double w = bounds.Width; // Width (Max.X - Min.X)
double h = bounds.Height; // Height (Max.Y - Min.Y)
VPoint c = bounds.Center; // Center point
double a = bounds.Area; // Width * Height

// Methods
bool inside = bounds.Contains(point); // Point containment
bool overlaps = bounds.Intersects(other); // Intersection test
BoundingBox combined = bounds.Union(other); // Combine bounds
BoundingBox bigger = bounds.Expand(10); // Expand by distance

// Tuple deconstruction (backwards compatible)
var (minPt, maxPt) = shape.GetBounds();
```

ICurve Interface

Shapes that represent curves (VLine, VCircle, VArc, VEllipse, VPolyline, VBezier, VSpline) implement the **ICurve** interface. Since **ICurve** extends **IDrawable**, all curves can be drawn directly:

```
// Work with curves generically
ICurve curve = new VLine(0, 0, 100, 50);
curve.Draw(); // ICurve extends IDrawable

// Curve operations
VPoint start = curve.StartPoint;
VPoint end = curve.EndPoint;
List<VPoint> vertices = curve.Vertices; // Key vertices/control points
double length = curve.GetLength();

// Divide curve into segments
List<VPoint> points = curve.Divide(10); // 11 points (including start/end)

// Measure points at fixed intervals
List<VPoint> measured = curve.Measure(25); // Points every 25 units

// Project a point onto the curve
VPoint closest = curve.Project(new VPoint(50, 50));

// Get point at specific distance along curve
VPoint midPoint = curve.PointAtSegmentLength(length / 2);

// Get point at normalized parameter (0 to 1)
VPoint quarterPoint = curve.PointAtParameter(0.25); // 25% along the curve

// Get parameter for a point on the curve (inverse of PointAtParameter)
double param = curve.ParameterAtPoint(quarterPoint); // Returns ~0.25

// Create offset curve
ICurve offset = curve.Offset(10);
offset.Draw();

// Split curve at a point
var (first, second) = curve.SplitAtPoint(midPoint);

// Get normal vector at a point
VXYZ normal = curve.NormalAtPoint(midPoint);

// Check if curve is self-intersecting
bool selfIntersects = curve.SelfIntersecting;

// Intersect with another curve
IntersectionResult result = curve.Intersect(otherCurve);
if (result.HasIntersection)
{
    foreach (var pt in result.Points)
        new VPoint(pt.X, pt.Y).Draw(); // Draw intersection points
}
```


Curve Intersection

All `ICurve` types support intersection detection:

```
var line1 = new VLine(0, 0, 100, 100);
var line2 = new VLine(0, 100, 100, 0);
var circle = new VCircle(50, 50, 30);

// Line-Line intersection
var result = line1.Intersect(line2);
if (result.IsSinglePoint)
    VizConsole.Log($"Lines cross at: {result.Points[0]}");

// Line-Circle intersection (may have 0, 1, or 2 points)
var circleResult = line1.Intersect(circle);
VizConsole.Log($"Found {circleResult.Points.Count} intersections");

// Check for overlapping segments (collinear lines)
if (result.HasOverlap)
    foreach (var overlapCurve in result.Curves)
        overlapCurve.Draw();
```

Self-Intersection Detection

The `SelfIntersecting` property indicates whether a curve crosses itself:

```
// Simple curves are never self-intersecting
var line = new VLine(0, 0, 100, 100);
VizConsole.Log($"Line self-intersects: {line.SelfIntersecting}"); // false

// Complex curves may self-intersect
var polyline = new VPolyline(
    new VPoint(0, 0),
    new VPoint(100, 0),
    new VPoint(50, 50),
    new VPoint(50, -50) // crosses back over
);
VizConsole.Log($"Polyline self-intersects: {polyline.SelfIntersecting}"); // true
```

Example: Complete Drawing

```
using Code2Viz.Geometry;
using System;

namespace StartViz
{
    public class Viz
```

```
{
    public static void Main()
    {
        // Set global styling
        ShapeDefaults.GlobalColor = "Cyan";
        ShapeDefaults.GlobalStrokeThickness = 2;

        // Draw coordinate axes
        new VArrow(-150, 0, 150, 0).Draw(); // X-axis
        new VArrow(0, -150, 0, 150).Draw(); // Y-axis

        // Draw a house
        var house = new VPolygon(
            new VPoint(-50, -50),
            new VPoint(50, -50),
            new VPoint(50, 30),
            new VPoint(0, 70),
            new VPoint(-50, 30)
        );
        house.FillColor = "#40FFFF00";
        house.Draw();

        // Door
        var door = new VRectangle(-15, -50, 30, 50);
        door.FillColor = "#80804000";
        door.Draw();

        // Window
        var window = new VCircle(25, 0, 15);
        window.FillColor = "#8000FFFF";
        window.Draw();

        // Sun
        var sun = new VCircle(100, 100, 25);
        sun.Color = "Yellow";
        sun.FillColor = "#80FFFF00";
        sun.Draw();

        // Sun rays
        for (int i = 0; i < 8; i++)
        {
            double angle = i * 45 * Math.PI / 180;
            double x1 = 100 + 35 * Math.Cos(angle);
            double y1 = 100 + 35 * Math.Sin(angle);
            double x2 = 100 + 50 * Math.Cos(angle);
            double y2 = 100 + 50 * Math.Sin(angle);
            var ray = new VLine(x1, y1, x2, y2);
            ray.Color = "Yellow";
            ray.Draw();
        }

        VizConsole.Log("House drawing complete!");
    }
}
```

```
}  
}
```

Building and Running

Prerequisites

- .NET 9.0 SDK
- Windows (WPF application)

Build

```
cd Code2Viz  
dotnet restore  
dotnet build
```

Run

```
dotnet run
```

Dependencies

- **AvalonEdit** (6.3.0.90) - Code editor with syntax highlighting
- **Microsoft.CodeAnalysis.CSharp** (4.8.0) - Roslyn compilation for C# code execution
- **FSharp.Compiler.Service** - F# compilation support
- **NuGet.Protocol** - Package management integration

Getting Help

- **Built-in Help:** Help > API Reference (F1) opens comprehensive documentation
- **Welcome Page:** The Help window shows a getting-started guide by default
- **Console Output:** Use `VizConsole.Log()` for debugging

License

This project is for experimental/educational purposes.